Complete Wireshark Filters Guide

1. Filter `http.accept_language`

What is it specifically used for?

- **Geolocation analysis**: Identify geographic origin of users
- **Web application troubleshooting**: Detect localization issues
- **Security analysis**: Identify anomalous patterns in requests
- **CDN optimization**: Analyze language distribution for server configuration
- **Bot detection**: Identify automated traffic through Accept-Language patterns

Basic syntax

```
http.accept_language
http.accept_language contains "es"
http.accept_language == "en-US,en;q=0.9"
```

Building effective filters

1. **Basic filter**: `http.accept_language` - Shows all HTTP requests with Accept-Language header
2. **Specific filter**: `http.accept_language contains "es-ES"` - Only Spanish from Spain requests
3. **Combined**: `http.accept_language and http.host == "example.com"` - Filter by language and host

When to use this filter

- Investigating localization issues in web applications
- Analyzing traffic patterns by geographic region
- Detecting anomalous behavior in HTTP requests
- Optimizing content by language

Optimization

- Combine with IP filters to reduce results: `http.accept_language and ip.src == 192.168.1.0/24`
- Use regular expressions for multiple languages: `http.accept_language matches "es|pt|it"`

Common errors to avoid

- Don't confuse with `http.content_language` (response vs request)

- Remember that some browsers send multiple languages with priorities
- Don't assume language reflects actual user location

---

2. Filter `dhcp`

What is it specifically used for?

- **Connectivity troubleshooting**: Diagnose IP assignment issues
- **Network auditing**: Monitor DHCP assignments and renewals
- **Security analysis**: Detect DHCP spoofing or rogue servers
- **Performance optimization**: Analyze DHCP response times
- **Inventory management**: Track devices by MAC addresses

Basic syntax

```
dhcp
bootp
dhcp.option.dhcp == 1   # DHCP Discover
dhcp.option.dhcp == 2   # DHCP Offer
dhcp.option.dhcp == 3   # DHCP Request
dhcp.option.dhcp == 5   # DHCP ACK
```

Building effective filters

1. **All DHCP messages**: `dhcp`
2. **Only discovers**: `dhcp.option.dhcp == 1`
3. **Renewal issues**: `dhcp.option.dhcp == 3 and dhcp.option.dhcp == 7`
4. **Specific device**: `dhcp and eth.addr == aa:bb:cc:dd:ee:ff`

When to use each variant

- **Initial troubleshooting**: Use generic `dhcp` to see entire conversation
- **Connectivity issues**: Filter by specific message type
- **Security auditing**: Look for multiple offers or unauthorized servers
- **Performance analysis**: Measure time between Discover and ACK

Optimization for large volumes

- Filter by network segment: `dhcp and ip.src_host in {192.168.1.0/24}`

- Use specific time ranges instead of complete captures
- Combine with VLAN filters if necessary

Common errors

- Confusing DHCP with BOOTP (use `bootp` for very basic captures)
- Not considering DHCP relay agents in complex networks
- Forgetting that DHCP uses broadcast, affecting analysis

---

3. Filter `nbns`

What is it specifically used for?

- **Windows network troubleshooting**: Resolve NetBIOS name resolution issues
- **Security analysis**: Detect NetBIOS name spoofing attacks
- **Legacy network auditing**: Monitor traffic from older Windows systems
- **SMB diagnostics**: Identify issues prior to SMB connections
- **Network mapping**: Discover active Windows hosts

Basic syntax

```
nbns
nbns.name contains "WORKGROUP"
nbns.type == 0x20    # File Server Service
nbns.flags.response == 1    # Responses only
```

Building effective filters

1. **General traffic**: `nbns`
2. **Specific queries**: `nbns.flags.response == 0`
3. **By name**: `nbns.name contains "SERVER01"`
4. **By service type**: `nbns.type == 0x00` (Workstation Service)

When to use this filter

- Diagnosing "Network Neighborhood" issues in Windows
- Investigating slowness in shared resource access
- Analyzing name propagation in workgroups

- Detecting NetBIOS name conflicts

Optimization

- Combine with time filters for specific events
- Filter by subnet to reduce noise: `nbns and ip.addr == 192.168.1.0/24`
- Use together with SMB filters for complete analysis

Common errors

- Confusing with DNS queries (they are different protocols)
- Not considering that NetBIOS over TCP/IP may use port 137
- Forgetting that modern systems may not use NetBIOS

---

4. Filter `kerberos.CNameString`

What is it specifically used for?

- **Authentication analysis**: Track specific login attempts
- **SSO troubleshooting**: Diagnose Single Sign-On issues
- **Security auditing**: Monitor access and detect anomalies
- **Forensic analysis**: Investigate specific user activity
- **AD optimization**: Analyze authentication patterns

Basic syntax

```
kerberos.CNameString
kerberos.CNameString == "user01"
kerberos.CNameString contains "@DOMAIN.COM"
kerberos.realm == "COMPANY.LOCAL"
```

Building effective filters

1. **Specific user**: `kerberos.CNameString == "john.doe"`
2. **Specific domain**: `kerberos.CNameString contains "@CORP.COM"`
3. **With errors**: `kerberos.CNameString and kerberos.error_code`
4. **Combined with IP**: `kerberos.CNameString == "admin" and ip.src == 192.168.1.100`

When to use this filter

- Investigating authentication issues for specific users
- Tracking suspicious activity of administrative accounts
- Diagnosing trust issues between domains
- Analyzing access patterns for optimization

Optimization

- Combine with time filters for specific windows
- Use with `kerberos.error_code` to quickly identify failures
- Filter by specific domain controller if necessary

Common errors

- Not distinguishing between pre-authentication and complete authentication
- Confusing CNameString with SNameString (client vs service)
- Not considering UPN vs SAM account name format

---

5. Filter `ldap contains "CN=Users"`

What is it specifically used for?

- **Active Directory auditing**: Monitor queries to user containers
- **Application troubleshooting**: Diagnose LDAP binding issues
- **Security analysis**: Detect unauthorized user enumeration
- **Query optimization**: Analyze LDAP query efficiency
- **Compliance**: Track access to sensitive user information

Basic syntax

```
ldap contains "CN=Users"
ldap.basedn contains "CN=Users"
ldap.filter contains "objectClass=user"
ldap.resultCode != 0  # Errors only
```

Building effective filters

1. **Basic queries**: `ldap contains "CN=Users"`
2. **With specific filters**: `ldap contains "CN=Users" and ldap.filter contains "sAMAccountName"`

3. **By application**: `ldap contains "CN=Users" and ip.src == 192.168.1.50`
4. **Errors only**: `ldap contains "CN=Users" and ldap.resultCode != 0`

When to use this filter

- Diagnosing application authentication issues
- Investigating slow or inefficient AD queries
- Auditing access to user information
- Detecting LDAP enumeration attacks

Optimization for large volumes

- Combine with source IP filters for specific applications
- Use time ranges for targeted analysis
- Filter by response code to focus on issues

Common errors

- Not distinguishing between read and modification queries
- Confusing CN=Users container with custom user OUs
- Not considering paginated queries in large environments

Combined Practical Use Cases

Comprehensive Security Analysis

```
(dhcp and dhcp.option.dhcp == 1) or
(nbns and nbns.flags.response == 0) or
(kerberos.CNameString and kerberos.error_code) or
(ldap contains "CN=Users" and ldap.resultCode != 0)
```

Windows Connectivity Troubleshooting

```
dhcp or nbns or (kerberos.CNameString == "problemuser") or
(ldap contains "CN=Users" and ip.src == 192.168.1.100)
```

Network Performance Analysis

```
(http.accept_language and http.time > 2.0) or
```

```
(dhcp and frame.time_delta > 1.0) or
(ldap contains "CN=Users" and ldap.time > 0.5)
```

General Best Practices

1. **Use incremental filtering**: Start with broad filters and progressively refine
2. **Combine related protocols**: DHCP + NetBIOS + Kerberos for complete Windows analysis
3. **Include temporal context**: Use `frame.time_relative` for sequence analysis
4. **Document complex filters**: Save custom filters for reuse
5. **Validate results**: Verify filters capture all relevant cases

Complementary Tools

- **Protocol statistics**: To validate volumes before filtering
- **Time graphs**: To identify temporal patterns
- **Selective export**: For further analysis with specialized tools
- **Coloring**: For quick visual pattern identification